

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Programa de Ingeniería de Sistemas y Computación
HPC

Multiplicación de matrices de forma secuencial y concurrente

Presentado por:

Juan Esteban Cardona Blandón
Jhoan Esteban Corrales Rodríguez
Sebastian Perdomo
Juana Valentina Martínez

Profesor:

Ramiro Andrés Barrios Valencia

Pereira, Colombia

16 de marzo de 2026

Índice

1. Introducción	2
2. Marco Teórico	3
2.1. Computación de Alto Rendimiento (HPC)	3
2.2. Fundamentos Matemáticos de la Multiplicación Matricial	3
2.3. Complejidad Computacional de la Multiplicación de Matrices	4
2.4. Programación Secuencial y Concurrente	4
2.5. Sistemas de Memoria Compartida	4
2.6. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria	5
2.7. Optimización Algorítmica y Aprovechamiento de la Caché	5
2.8. Optimización a Nivel de Compilador	6
2.9. Métricas de Rendimiento en Computación Secuencial	6
2.10. Métricas de Rendimiento en Computación Paralela	7
3. Marco Conceptual	8
3.1. Características del PC	8
3.2. Características del software	9
4. Desarrollo	9
4.1. Implementación Secuencial	9
4.2. Implementación Concurrente con Hilos	9
4.3. Implementación Paralela con Procesos	10
4.4. Calculo del tiempo de ejecución	10
5. Pruebas	10
5.1. Resultados del algoritmo secuencial	11
5.1.1. Optimización del código	11
5.2. Resultados algoritmo concurrente por hilos	12
5.2.1. Speedup	15
5.3. Resultados algoritmo concurrente por procesos	17
5.3.1. Resultados del algoritmo secuencial optimizado por cache line	19
6. Comparaciones entre algoritmos	22
7. Conclusiones	24

1. Introducción

El continuo avance de las arquitecturas de hardware ha impulsado la evolución de la Computación de Alto Rendimiento (HPC), permitiendo resolver problemas matemáticos de dimensiones masivas en tiempos computacionalmente viables. Un exponente fundamental de estos desafíos es la multiplicación de matrices, una operación algebraica cuya complejidad de $\mathcal{O}(n^3)$ impone una carga de procesamiento extrema a medida que crecen las dimensiones de los datos. A pesar de la aparente simplicidad de su implementación iterativa básica, la optimización de este algoritmo es un área de constante investigación, dado que constituye el núcleo estructural de aplicaciones modernas críticas, tales como el álgebra lineal computacional, la simulación física y el entrenamiento de modelos de inteligencia artificial (Dongarra et al., 1990).

Históricamente, el aumento de la frecuencia de reloj en los microprocesadores permitía acelerar la ejecución de algoritmos secuenciales de forma directa. Sin embargo, tras alcanzar los límites físicos y térmicos del silicio, la industria se desplazó hacia arquitecturas multinúcleo que exigen un paradigma de programación paralela (Patterson & Hennessy, 2017). Bajo este enfoque, el diseño de software eficiente requiere distribuir la carga de trabajo de manera concurrente. Para lograrlo en el lenguaje C, los sistemas operativos ofrecen distintos mecanismos, destacando la paralelización ligera mediante hilos (compartiendo el mismo espacio de memoria) y la paralelización pesada mediante la creación de procesos independientes; cada uno con implicaciones directas sobre el rendimiento global y el consumo de recursos (Silberschatz et al., 2018).

El presente trabajo tiene como objetivo principal analizar, comparar y documentar el desempeño del algoritmo de multiplicación de matrices bajo tres enfoques de programación: una implementación secuencial, una concurrente basada en hilos mediante la librería Pthreads, y una paralela basada en la duplicación de procesos mediante la llamada al sistema `fork`. A través de experimentación empírica variando el tamaño de las matrices y la cantidad de unidades de procesamiento asignadas, se medirán los tiempos reales de ejecución. Finalmente, apoyándose en métricas estandarizadas como el *Speedup* (aceleración) (Hager & Wellein, 2010), se busca comprobar matemáticamente la eficacia de la paralelización para reducir los tiempos de cómputo, así como evidenciar los cuellos de botella generados por la gestión de memoria y el límite de los recursos físicos del hardware.

2. Marco Teórico

2.1. Computación de Alto Rendimiento (HPC)

La Computación de Alto Rendimiento (HPC, por sus siglas en inglés) consiste en la agregación de potencia de cálculo a través de arquitecturas paralelas para resolver problemas científicos y tecnológicos de extrema complejidad. Esta disciplina permite ejecutar algoritmos y modelados matemáticos que resultarían inabarcables para un sistema tradicional debido a restricciones de tiempo y recursos de hardware (Hager & Wellein, 2010). Al dividir cargas masivas de trabajo en instrucciones más pequeñas que se procesan simultáneamente, HPC maximiza la velocidad de procesamiento de datos y la obtención de resultados en la ingeniería (Patterson & Hennessy, 2017).

2.2. Fundamentos Matemáticos de la Multiplicación Matricial

La multiplicación de matrices es una operación algebraica bidimensional fundamental que procesa dos matrices de origen para producir una tercera. Dadas dos matrices A de dimensión $m \times p$ y B de dimensión $p \times n$, su producto da como resultado una matriz C de dimensiones $m \times n$. Matemáticamente, el valor de cada celda individual C_{ij} en la matriz resultante se calcula mediante el producto punto o escalar entre la i -ésima fila de la matriz A y la j -ésima columna de la matriz B . Esta operación se expresa de forma analítica mediante la siguiente sumatoria:

$$C_{ij} = \sum_{k=1}^p A_{ik} B_{kj}$$

Para que esta operación esté matemáticamente definida y sea computacionalmente viable, existe una condición estricta de dimensionalidad: el número de columnas de la primera matriz (p) debe coincidir exactamente con el número de filas de la segunda matriz (Strang, 2016).

En el contexto de la ingeniería y la Computación de Alto Rendimiento (HPC), la multiplicación matricial trasciende la teoría pura para consolidarse como la rutina principal (*kernel*) subyacente en la gran mayoría del software científico. Esta operación constituye el pilar del Nivel 3 de la especificación BLAS (*Basic Linear Algebra Subprograms*), el estándar de facto para operaciones de álgebra lineal. La eficiencia con la que un procesador ejecuta este algoritmo dicta directamente el rendimiento de aplicaciones críticas modernas, tales como el modelado de dinámica de fluidos, la física de partículas, el procesamiento masivo de imágenes y, muy especialmente, el entrenamiento de arquitecturas profundas en inteligencia artificial mediante redes neuronales (Dongarra et al., 1990).

2.3. Complejidad Computacional de la Multiplicación de Matrices

La multiplicación de matrices es un problema canónico en el álgebra lineal computacional, utilizado frecuentemente para evaluar el desempeño y la robustez de las arquitecturas de hardware. Dadas dos matrices cuadradas A y B de dimensiones $n \times n$, el algoritmo iterativo secuencial clásico exige iterar mediante tres ciclos anidados, realizando matemáticamente n^3 multiplicaciones y $n^2(n - 1)$ sumas (Cormen et al., 2009).

Esto establece una complejidad temporal asintótica estricta de $\mathcal{O}(n^3)$. Este crecimiento cúbico implica que, si el tamaño de la matriz se duplica, el esfuerzo computacional se multiplica por un factor de ocho. Debido a esta pronunciada curva de costo algorítmico, la distribución de la carga mediante técnicas de programación paralela (hilos o procesos) se vuelve un requisito indispensable cuando se operan matrices de dimensiones elevadas (Grama et al., 2003).

2.4. Programación Secuencial y Concurrente

La programación secuencial es el paradigma clásico de desarrollo donde las instrucciones de un algoritmo se ejecutan de forma estrictamente lineal, una tras otra, sobre un único núcleo de procesamiento. En este modelo tradicional, el límite de rendimiento está directamente dictado por la frecuencia de reloj del hardware. Como respuesta a estas limitaciones físicas, la programación concurrente estructura el software en múltiples hilos o tareas independientes que hacen progreso de forma superpuesta en el tiempo. Cuando estas tareas operan de manera simultánea en una arquitectura de hardware multinúcleo, la concurrencia se traduce en paralelismo físico, lo cual permite dividir la carga de trabajo y reducir drásticamente el tiempo de ejecución en problemas de cómputo intensivo (Ben-Ari, 2006).

2.5. Sistemas de Memoria Compartida

En el ámbito de la computación paralela, el modelo de memoria compartida establece que múltiples hilos de ejecución (como los creados por la librería Pthreads) operan dentro de un mismo espacio de direccionamiento lógico. A nivel de arquitectura, esto significa que todos los núcleos del procesador pueden acceder de forma directa y simultánea a las mismas estructuras de datos residentes en la memoria principal. Para problemas matriciales, este modelo resulta excepcionalmente eficiente frente a los sistemas de memoria distribuida, ya que evita la necesidad de copiar y transmitir los datos de las matrices de origen repetidamente entre procesos. Aunque este paradigma exige control riguroso para evitar condiciones de carrera, en operaciones intrínsecamente independientes como el cálculo individual de las celdas de una

matriz resultado, su implementación maximiza la eficiencia computacional sin sobrecarga de sincronización (Silberschatz et al., 2018).

2.6. Jerarquía de Caché, Líneas de Caché y Localidad de Memoria

El rendimiento de una aplicación de alto rendimiento (HPC) no depende exclusivamente de la capacidad matemática del procesador, sino también del acceso al subsistema de memoria. Para mitigar la alta latencia térmica y física de la memoria RAM, los procesadores modernos integran una jerarquía de memoria ultrarrápida cercana a los núcleos, conocida como memoria Caché, habitualmente dividida en niveles (L1, L2 y L3). La unidad mínima de transferencia de información entre la RAM principal y la memoria caché no es un byte aislado, sino un bloque continuo de datos denominado línea de caché (*cache line*), que en las arquitecturas modernas suele constar de 64 bytes (Drepper, 2007).

Esta transferencia por bloques está fundamentada en el principio de localidad espacial de memoria, un concepto empírico que dicta que si un programa accede a una dirección de memoria, es inminentemente probable que acceda a las direcciones adyacentes a continuación. En lenguajes de programación como C, las matrices bidimensionales se almacenan en la memoria RAM de manera contigua organizadas por filas (*row-major order*). Durante la multiplicación matricial clásica iterativa, la segunda matriz se recorre inevitablemente por columnas; este patrón de acceso ortogonal rompe por completo la localidad espacial. Al intentar leer datos saltando grandes bloques de memoria, los valores necesarios no se encuentran en la línea de caché cargada, provocando fallos de caché (*cache misses*) que obligan al procesador a detenerse (*stall*) mientras busca los datos nuevamente en la lenta memoria RAM, lo cual penaliza severamente el desempeño del algoritmo (Patterson & Hennessy, 2017).

2.7. Optimización Algorítmica y Aprovechamiento de la Caché

Aunque la paralelización distribuye la carga de trabajo, el cuello de botella de la memoria solo puede resolverse reestructurando el algoritmo subyacente. La estrategia de optimización más directa para la multiplicación de matrices en C consiste en el intercambio de bucles (*loop interchange*). Altera el orden tradicional de los ciclos anidados de i, j, k a i, k, j . Matemáticamente, el resultado es idéntico, pero a nivel de hardware, el bucle más interno ahora recorre las matrices B y C estrictamente por filas. Esto garantiza que el procesador consuma por completo los 64 bytes de cada línea de caché (*cache line*) cargada antes de solicitar la siguiente, aprovechando al máximo la localidad espacial y reduciendo las penalizaciones por fallos de caché en órdenes de magnitud (Lam et al., 1991).

Para matrices masivas que exceden la capacidad total de la memoria caché (L1 y L2),

la literatura de HPC emplea una técnica avanzada denominada multiplicación por bloques o *Loop Tiling*. Este método particiona las matrices originales en submatrices (o bloques) de un tamaño cuidadosamente calculado para que residan por completo dentro de la caché rápida del procesador. En lugar de multiplicar filas y columnas completas, el algoritmo opera sobre estos bloques aislados, reutilizando los datos cargados repetidas veces antes de que el controlador de memoria los descarte (*eviction*). Esta técnica transforma un problema limitado por el ancho de banda de la memoria (*memory-bound*) en uno puramente computacional (*compute-bound*) (Grama et al., 2003).

2.8. Optimización a Nivel de Compilador

El desarrollo en lenguajes de bajo nivel como C delega un porcentaje crítico del rendimiento final al compilador (como GCC o Clang). La optimización por compilador consiste en un conjunto de transformaciones semánticas que el traductor aplica sobre el código fuente para generar un código máquina altamente eficiente, sin alterar la lógica del programa. Mediante el uso de banderas de optimización agresivas (típicamente `-O2` o `-O3`), el compilador realiza análisis de flujo de datos para aplicar mejoras automáticas como la eliminación de código muerto y la propagación de constantes (Allen & Kennedy, 2001).

Para algoritmos intensivos iterativos como la multiplicación matricial, dos optimizaciones de compilador son vitales. La primera es el desenrollado de bucles (*loop unrolling*), que reduce la sobrecarga de evaluar repetidamente la condición de parada del bucle replicando el cuerpo de las instrucciones. La segunda es la autovectorización, donde el compilador detecta operaciones matemáticas independientes y las agrupa utilizando instrucciones SIMD (*Single Instruction, Multiple Data*). Con banderas como `-march=native`, el compilador explota las extensiones vectoriales específicas del hardware local (como AVX2 o AVX-512 en procesadores Intel/AMD), permitiendo multiplicar múltiples pares de números de punto flotante en un único ciclo de reloj, complementando perfectamente el paralelismo de hilos de alto nivel (Patterson & Hennessy, 2017).

2.9. Métricas de Rendimiento en Computación Secuencial

El análisis de un algoritmo secuencial requiere establecer una línea base absoluta sobre la cual evaluar cualquier mejora algorítmica o de hardware. La métrica fundamental empírica es el Tiempo de Ejecución (*Wall-clock time*), que representa el tiempo físico real transcurrido desde el inicio hasta la finalización del programa. Sin embargo, dado que este valor fluctúa dependiendo de la interferencia del sistema operativo y los accesos a memoria, la arquitectura de computadores recurre a métricas de rendimiento intrínseco (*throughput*) (Patterson &

Hennessy, 2017).

Para cargas de trabajo de alto rendimiento algebraico como la multiplicación de matrices, la métrica estandarizada predominante es el volumen de Operaciones de Punto Flotante por Segundo (FLOPS, por sus siglas en inglés). Esta medida cuantifica el rendimiento matemático bruto del procesador y se define mediante la siguiente relación:

$$\text{FLOPS} = \frac{\text{Operaciones Aritméticas Totales}}{T_{\text{ejecución}}} \quad (1)$$

Para el algoritmo clásico iterativo que multiplica dos matrices cuadradas de dimensión $n \times n$, el número total de operaciones de punto flotante se establece analíticamente en $2n^3$ (contabilizando una instrucción de multiplicación y una de suma por cada iteración del bucle más interno). Al contrastar los FLOPS experimentales obtenidos en la ejecución secuencial contra el rendimiento teórico máximo (*Peak FLOPS*) que el fabricante del procesador garantiza para un solo núcleo, es posible diagnosticar ineficiencias de memoria o cuellos de botella antes de recurrir a la paralelización (Hager & Wellein, 2010).

Adicionalmente, el rendimiento de la ejecución en un único núcleo se evalúa a nivel microarquitectónico mediante el IPC (Instrucciones por Ciclo de reloj). Esta métrica indica la eficiencia con la que el código compilado logra mantener ocupada la segmentación (*pipeline*) del procesador, minimizando las detenciones o burbujas producidas por dependencias de datos en la ejecución lineal (Yi et al., 2020).

2.10. Métricas de Rendimiento en Computación Paralela

Para cuantificar el éxito de una implementación paralela y su eficiencia frente a la ejecución secuencial, la literatura científica emplea métricas matemáticas estandarizadas. La métrica fundamental es el *Speedup* o Aceleración (S_p), la cual mide la ganancia de velocidad relativa. Se define formalmente como:

$$S_p = \frac{T_1}{T_p} \quad (2)$$

donde T_1 representa el tiempo de ejecución del algoritmo secuencial óptimo sobre un único núcleo, y T_p es el tiempo de ejecución del algoritmo paralelo empleando p unidades de procesamiento (hilos o procesos). Derivada de esta métrica se encuentra la Eficiencia (E_p), que evalúa el grado de aprovechamiento de los recursos físicos, indicando qué fracción del tiempo los procesadores realizan trabajo útil sin quedar ociosos:

$$E_p = \frac{S_p}{p} = \frac{T_1}{p \cdot T_p} \quad (3)$$

El desempeño asintótico de estos sistemas está regido por dos leyes analíticas que abordan la escalabilidad desde diferentes perspectivas teóricas. La **Ley de Amdahl** evalúa la *escalabilidad fuerte*, demostrando que el *Speedup* máximo de un sistema está estrictamente limitado por la fracción de código que no se puede paralelizar. Asumiendo un tamaño de problema fijo, la ley se expresa como:

$$S_{\text{Amdahl}} \leq \frac{1}{(1 - f) + \frac{f}{p}} \quad (4)$$

donde f es la fracción del programa que es paralelizable y $(1 - f)$ es la porción estrictamente secuencial (como la inicialización de memoria mediante `malloc` o la lectura/escritura en disco). Según este postulado matemático, incluso si el número de procesadores p tiende a infinito, la aceleración máxima nunca superará $1/(1 - f)$. Esto explica el rendimiento decreciente observado cuando el uso de múltiples hilos lógicos en un procesador alcanza su límite físico y no aporta mejoras significativas (Patterson & Hennessy, 2017).

En contraste, la **Ley de Gustafson-Barsis** describe la *escalabilidad débil*. Esta ley argumenta que, en la práctica profesional, los ingenieros no buscan resolver un problema fijo más rápido, sino que utilizan el mayor poder de cómputo para resolver problemas de un tamaño sustancialmente mayor en el mismo marco de tiempo (Gustafson, 1988). Su modelo se define como:

$$S_{\text{Gustafson}} = (1 - f) + p \cdot f \quad (5)$$

Bajo la visión de Gustafson, a medida que la dimensión de las matrices crece de forma masiva, la cantidad de operaciones paralelas f (multiplicaciones flotantes) aumenta drásticamente, haciendo que la fracción secuencial inicial $(1 - f)$ pierda peso relativo. Esto mitiga el cuello de botella pronosticado por Amdahl, permitiendo alcanzar un *Speedup* escalable y un uso altamente eficiente de los recursos al ajustar la carga de trabajo proporcionalmente al hardware.

3. Marco Conceptual

3.1. Características del PC

- Procesador: Ryzen 5 7600X
- Cantidad de núcleos: 6
- Cantidad de subprocesos/hilos: 12

- Frecuencia básica del procesador: 4.7GHz
- Frecuencia Turbo máxima: 5.3GHz
- Memoria RAM: 32GB DDR5 @ 5200 MHz
- SSD: 1TB - R: 3500 MB/s - W: 1900 MB/s
- Sistema operativo: Arch Linux

3.2. Características del software

Se puede encontrar el código fuente del proyecto en el siguiente repositorio de GitHub:
<https://github.com/Juanes7222/HPC-Matrix.git>

4. Desarrollo

Habiendo establecido el marco teórico y los objetivos del presente trabajo, se procede a detallar la implementación de los tres enfoques de programación para la multiplicación de matrices: secuencial, concurrente con hilos y paralela con procesos. Cada sección describe la metodología seguida, las decisiones de diseño tomadas y los aspectos técnicos relevantes que influyen en el rendimiento de cada implementación.

4.1. Implementación Secuencial

La implementación secuencial de la multiplicación de matrices se basa en el algoritmo clásico de tres bucles anidados, que itera sobre las filas de la primera matriz y las columnas de la segunda matriz para calcular cada elemento del resultado. Este enfoque es directo y fácil de entender, pero su complejidad de $\mathcal{O}(n^3)$ lo hace ineficiente para matrices grandes. En esta implementación, se asigna memoria para las matrices utilizando arreglos dinámicos en C, y se inicializan con valores aleatorios para simular datos reales.

4.2. Implementación Concurrente con Hilos

La implementación concurrente con hilos se realiza utilizando la librería Pthreads, que permite crear múltiples hilos de ejecución dentro del mismo proceso. En este enfoque, se divide la tarea de multiplicación de matrices en bloques o filas, asignando cada bloque a un hilo diferente. Cada hilo calcula una parte del resultado de la multiplicación, y al finalizar, se sincronizan para asegurar que el resultado final esté completo. Este método aprovecha la

capacidad de los procesadores multinúcleo para mejorar el rendimiento, aunque introduce desafíos relacionados con la gestión de memoria compartida y la sincronización entre hilos.

4.3. Implementación Paralela con Procesos

La implementación paralela con procesos se lleva a cabo utilizando la llamada al sistema `fork`, que crea procesos independientes que no comparten memoria. En este enfoque, cada proceso se encarga de calcular una parte de la multiplicación de matrices, y los resultados se comunican a través de mecanismos de interproceso como tuberías o archivos temporales. Este método puede ofrecer un rendimiento mejorado en sistemas con múltiples núcleos, pero también introduce una sobrecarga significativa debido a la necesidad de comunicación entre procesos y la gestión de recursos del sistema operativo.

4.4. Calculo del tiempo de ejecución

Para medir el tiempo de ejecución de cada implementación, se utiliza la función `clock_gettime` de la biblioteca estándar de C, que proporciona una alta resolución temporal. Se registra el tiempo antes y después de la ejecución del algoritmo de multiplicación de matrices, y se calcula la diferencia para obtener el tiempo total de ejecución. Este proceso se repite varias veces para cada tamaño de matriz y número de hilos o procesos, con el fin de obtener un promedio representativo del rendimiento de cada enfoque.

5. Pruebas

En esta sección se presentan los resultados obtenidos a partir de la ejecución de las tres implementaciones de la multiplicación de matrices: secuencial, concurrente con hilos y paralela con procesos. Se describen los experimentos realizados, los datos recopilados y el análisis de los resultados en términos de tiempos de ejecución y métricas de rendimiento como el Speedup. Se probó el algoritmo para la multiplicación de matrices generando matrices cuadradas de dimensiones 400, 800, 1600, 3200 y 6400 respectivamente. El tiempo descrito en cada tabla se encuentra en milisegundos (ms). Se analizaron los distintos métodos evaluando los recursos consumidos como el tiempo; se sacaron algunas gráficas para mostrar los resultados obtenidos.

5.1. Resultados del algoritmo secuencial

5.1.1. Optimización del código

Para el algoritmo secuencial se usó una optimización por compilador con las banderas `-O3 -Wall -march=native -funroll-loops -flto -ffast-math -fomit-frame-pointer`, lo que permitió mejorar el rendimiento del código sin modificar su estructura. Esta optimización se tradujo en una reducción significativa del tiempo de ejecución, especialmente para matrices de mayor tamaño, al permitir al compilador aplicar técnicas avanzadas de optimización como la vectorización y el desenrollado de bucles.

El significado de cada bandera es el siguiente:

- **-O3:** Esta bandera habilita un nivel de optimización agresivo, permitiendo al compilador aplicar una amplia gama de técnicas para mejorar el rendimiento del código.
- **-Wall:** Activa la mayoría de las advertencias del compilador, lo que ayuda a identificar posibles problemas en el código que podrían afectar su rendimiento o funcionalidad.
- **-march=native:** Permite al compilador generar código optimizado para la arquitectura específica del procesador en el que se está compilando, aprovechando las características y capacidades de hardware disponibles.
- **-funroll-loops:** Habilita el desenrollado de bucles, lo que puede mejorar el rendimiento al reducir la sobrecarga de control de bucle y aumentar la paralelización de las instrucciones.
- **-flto:** Habilita la optimización a nivel de enlace, lo que permite al compilador realizar optimizaciones globales en todo el programa, incluso entre diferentes archivos de código fuente.
- **-ffast-math:** Permite al compilador realizar optimizaciones agresivas en operaciones matemáticas, lo que puede mejorar el rendimiento a costa de la precisión en algunos casos como la evaluación de funciones trigonométricas.
- **-fomit-frame-pointer:** Indica al compilador que no utilice un puntero de marco para las funciones, es decir, que no cree un marco de pila para cada función, lo que puede mejorar el rendimiento al reducir la cantidad de instrucciones necesarias para gestionar la pila de llamadas.

Estos fueron los tiempos de ejecución obtenidos para el algoritmo secuencial optimizado:

Secuencial con opt por compilador					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	30,005	312,955	2.540,758	36.022,321	434.910,671
2	30,071	312,064	2.540,398	35.590,382	344.113,604
3	30,091	312,066	2.550,371	35.602,554	365.954,583
4	30,147	312,059	2.543,867	35.751,773	334.690,484
5	30,083	312,649	2.541,859	35.526,842	341.364,101
6	30,026	311,893	2.546,591	35.525,175	510.874,781
7	30,062	312,047	2.541,431	35.557,655	505.512,386
8	30,237	311,989	2.543,702	35.225,193	502.152,475
9	30,240	311,605	2.539,963	35.261,713	325.713,787
10	30,227	311,890	2.542,086	35.340,683	325.443,173
Promedio	30,119	312,122	2.543,103	35.540,429	399.073,004
Desv. Est.	0,084	0,371	3,066	223,810	76.245,927
CV (%)	0,28	0,12	0,12	0,63	19,11

Figura 1: Tiempos de ejecución del algoritmo secuencial para diferentes tamaños de matriz.

5.2. Resultados algoritmo concurrente por hilos

Estos fueron los tiempos de ejecución obtenidos para el algoritmo concurrente con 2 hilos:

Concurrencia 2 hilos sin optimizar					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	78,584	701,624	5.570,639	63.560,000	481.391,665
2	77,480	701,230	5.720,959	63.517,097	564.437,274
3	77,151	701,110	5.568,161	63.254,180	563.552,750
4	76,873	690,388	5.574,338	63.383,972	564.276,365
5	78,184	701,433	5.567,155	63.546,728	559.626,947
6	78,150	685,945	5.572,233	63.471,746	565.896,408
7	77,179	688,399	5.580,680	63.163,276	496.146,357
8	77,788	685,586	5.681,222	62.512,786	469.160,875
9	77,288	688,526	5.565,958	63.334,870	562.554,659
10	78,463	688,811	5.567,370	63.331,956	564.297,173
Promedio	77,714	693,305	5.596,871	63.307,661	539.134,047
Desv. Est.	0,573	6,696	53,020	292,285	37.768,932
CV (%)	0,74	0,97	0,95	0,46	7,01

Figura 2: Tiempos de ejecución del algoritmo concurrente con 2 hilos para diferentes tamaños de matriz.

Estos fueron los tiempos de ejecución obtenidos para el algoritmo concurrente con 4 hilos:

Concurrencia 4 hilos sin optimizar					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	39,279	350,532	2.836,123	31.037,650	275.784,033
2	39,042	350,697	2.832,454	31.051,295	274.198,226
3	39,663	349,995	2.835,699	31.196,726	269.616,608
4	39,468	349,534	2.785,878	31.248,556	271.464,858
5	38,631	345,482	2.784,701	31.037,128	270.059,275
6	39,159	351,887	2.837,482	31.017,579	272.711,540
7	39,612	342,809	2.839,171	31.087,043	270.702,724
8	39,208	350,621	2.841,163	31.056,492	270.607,971
9	38,758	348,857	2.789,910	31.001,129	272.545,776
10	39,099	345,251	2.787,040	31.041,289	275.610,292
Promedio	39,192	348,567	2.816,962	31.077,489	272.330,130
Desv. Est.	0,319	2,835	24,683	76,581	2.127,290
CV (%)	0,81	0,81	0,88	0,25	0,78

Figura 3: Tiempos de ejecución del algoritmo concurrente con 4 hilos para diferentes tamaños de matriz.

Estos fueron los tiempos de ejecución obtenidos para el algoritmo concurrente con 6 hilos:

Concurrencia 6 hilos sin optimizar					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	27,177	231,425	1.925,535	19.635,070	181.488,392
2	27,103	231,266	1.898,067	20.025,583	183.978,726
3	27,247	231,623	1.901,463	19.810,115	184.183,954
4	27,055	232,311	1.898,751	19.818,645	185.783,860
5	26,886	231,015	1.900,755	20.160,773	183.293,908
6	26,954	231,519	1.897,427	20.028,125	181.376,852
7	27,357	231,937	1.888,214	19.926,920	184.224,754
8	26,932	231,396	1.905,779	19.848,389	183.341,570
9	26,819	231,102	1.901,984	19.797,740	185.132,488
10	27,137	231,006	1.899,542	19.979,232	183.477,022
Promedio	27,067	231,460	1.901,752	19.903,059	183.628,153
Desv. Est.	0,161	0,395	9,013	143,215	1.325,197
CV (%)	0,60	0,17	0,47	0,72	0,72

Figura 4: Tiempos de ejecución del algoritmo concurrente con 6 hilos para diferentes tamaños de matriz.

Estos fueron los tiempos de ejecución obtenidos para el algoritmo concurrente con 8 hilos:

Concurrencia 8 hilos sin optimizar					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	38,630	343,125	2.776,722	30.571,130	271.815,447
2	38,790	342,405	2.784,369	30.808,821	277.199,704
3	38,659	342,119	2.775,761	30.574,483	274.157,100
4	38,343	341,964	2.776,769	30.744,951	275.092,675
5	38,493	342,599	2.782,608	30.672,909	277.316,470
6	38,692	343,510	2.776,412	30.598,206	275.895,162
7	38,381	342,809	2.779,763	30.654,460	240.644,948
8	38,896	342,791	2.779,934	30.626,555	275.247,651
9	38,892	342,298	2.778,416	30.493,412	255.196,133
10	38,703	342,500	2.779,015	30.610,513	275.238,700
Promedio	38,648	342,612	2.778,977	30.635,544	269.780,399
Desv. Est.	0,183	0,442	2,664	85,982	11.497,849
CV (%)	0,47	0,13	0,10	0,28	4,26

Figura 5: Tiempos de ejecución del algoritmo concurrente con 8 hilos para diferentes tamaños de matriz.

Estos fueron los tiempos de ejecución obtenidos para el algoritmo concurrente con 12 hilos:

Concurrencia 12 hilos sin optimizar					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	27,135	241,861	1.930,823	20.183,973	226.602,819
2	27,523	239,888	1.896,905	20.253,136	232.067,245
3	27,098	241,346	1.885,993	20.032,987	189.432,855
4	27,452	240,599	1.894,744	20.011,600	186.765,113
5	27,533	240,538	1.936,175	19.980,713	188.875,706
6	27,471	241,950	1.905,557	20.052,402	189.134,831
7	27,396	243,316	1.907,132	20.210,726	187.229,086
8	27,489	243,256	1.897,400	20.171,908	186.405,010
9	27,182	240,299	1.905,931	20.018,515	185.067,773
10	27,074	240,382	1.900,956	19.892,782	186.407,894
Promedio	27,335	241,344	1.906,162	20.080,874	195.798,833
Desv. Est.	0,179	1,163	14,957	110,749	16.863,527
CV (%)	0,66	0,48	0,78	0,55	8,61

Figura 6: Tiempos de ejecución del algoritmo concurrente con 12 hilos para diferentes tamaños de matriz.

5.2.1. Speedup

1. Hilos sin opt						
Implementacion	N=400	N=800	N=1,600	N=3,200	N=6,400	Sp. prom.
Secuencial naive con O3_full	30.1 ms (ref)	312.1 ms (ref)	2,543.1 ms (ref)	35,540.4 ms (ref)	399,073.0 ms (ref)	1.00x
Concurrencia 2 hilos sin optimizar	77.7 ms 0.39x	693.3 ms 0.45x	5,596.9 ms 0.45x	63,307.7 ms 0.56x	539,134.0 ms 0.74x	0.52x
Concurrencia 4 hilos sin optimizar	39.2 ms 0.77x	348.6 ms 0.90x	2,817.0 ms 0.90x	31,077.5 ms 1.14x	272,330.1 ms 1.47x	1.04x
Concurrencia 6 hilos sin optimizar	27.1 ms 1.11x	231.5 ms 1.35x	1,901.8 ms 1.34x	19,903.1 ms 1.79x	183,628.2 ms 2.17x	1.55x
Concurrencia 8 hilos sin optimizar	38.6 ms 0.78x	342.6 ms 0.91x	2,779.0 ms 0.92x	30,635.5 ms 1.16x	269,780.4 ms 1.48x	1.05x
Concurrencia 12 hilos sin optimizar	27.3 ms 1.10x	241.3 ms 1.29x	1,906.2 ms 1.33x	20,080.9 ms 1.77x	195,798.8 ms 2.04x	1.51x

Figura 7: Speedup del algoritmo concurrente con diferentes cantidades de hilos para diferentes tamaños de matriz.

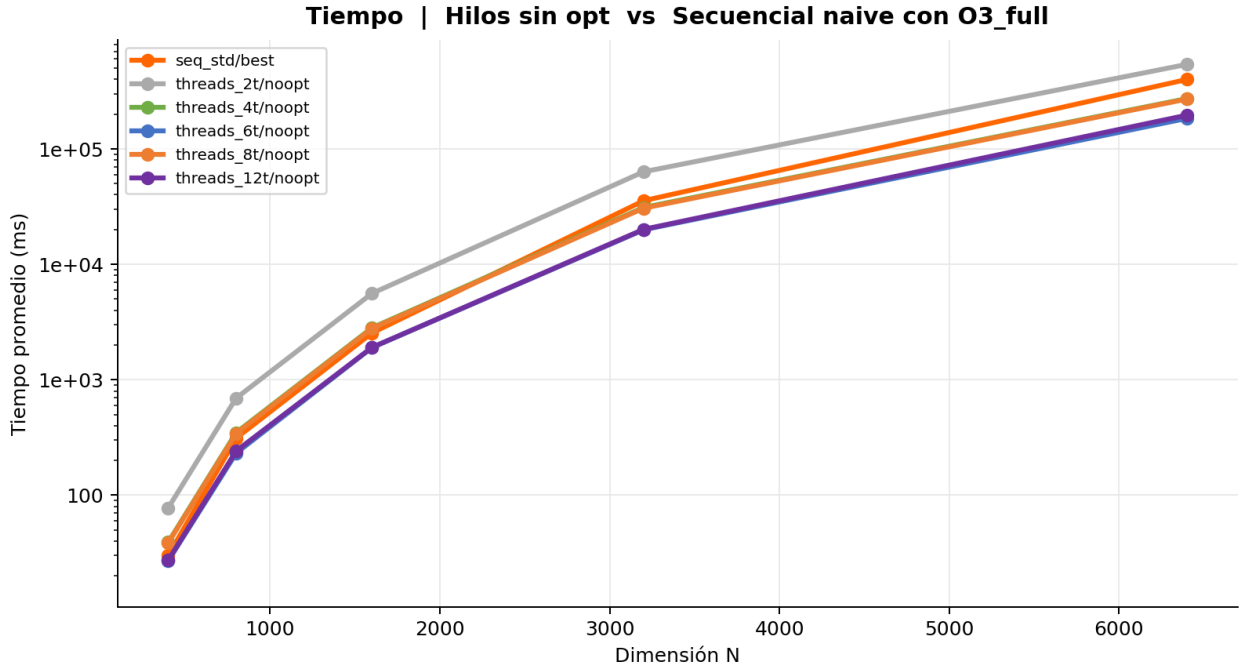


Figura 8: Speedup del algoritmo concurrente con diferentes cantidades de hilos para diferentes tamaños de matriz (zoom).

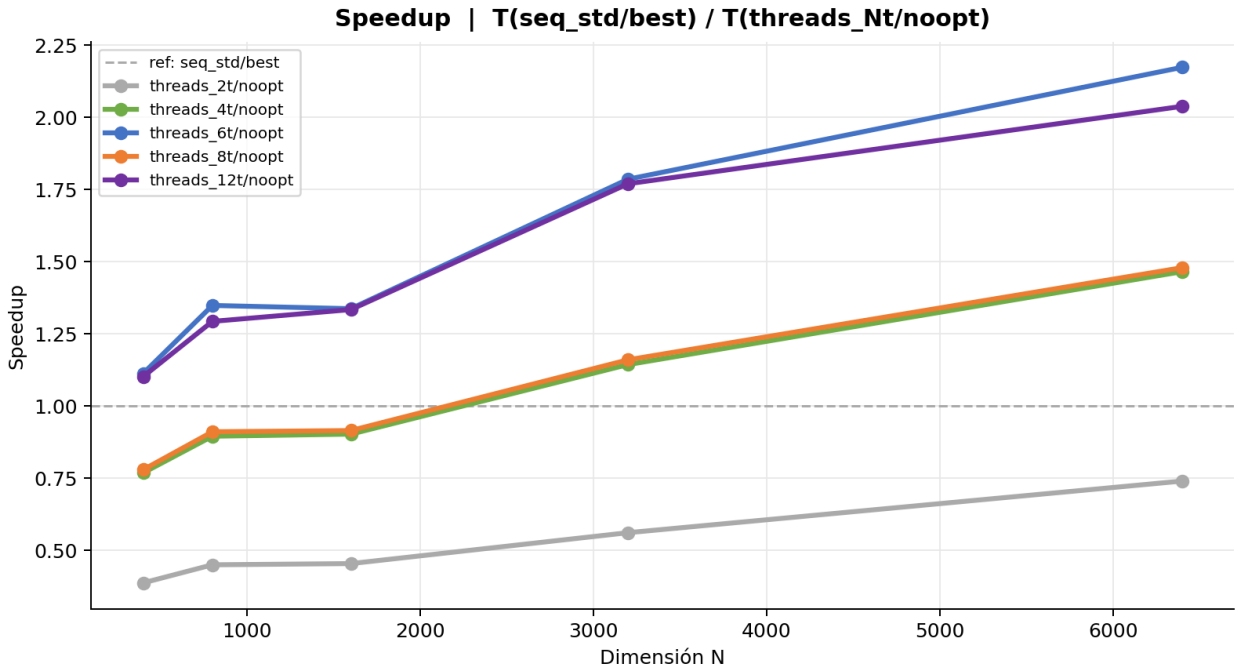


Figura 9: Speedup del algoritmo concurrente con diferentes cantidades de hilos para diferentes tamaños de matriz.

Como vemos en las figuras 7, 8 y 9, el algoritmo concurrente con hilos muestra un aumento en el rendimiento a medida que se incrementa la cantidad de hilos utilizados, especialmente para matrices de mayor tamaño. Sin embargo, el Speedup no es lineal debido a factores como la sobrecarga de gestión de hilos, la contención de recursos y la eficiencia del algoritmo paralelo. En algunos casos, se observa que el rendimiento se estabiliza o incluso disminuye al agregar más hilos, lo que sugiere que existe un punto óptimo en la cantidad de hilos para cada tamaño de matriz. Si observamos el gráfico de Speedup, podemos ver que para matrices con un tamaño mayor a 3000, el rendimiento mejora significativamente con a partir de los 4 hilos, pero con solo dos hilos el rendimiento es menor. Con 12 hilos el Speedup se estabiliza o disminuye ligeramente. Esto indica que la eficiencia del algoritmo paralelo depende en gran medida del tamaño de la matriz y la cantidad de hilos utilizados, y que existe un equilibrio entre la paralelización y la sobrecarga de gestión de hilos. En este caso el que obtuvo un mayor rendimiento fue utilizando 6 hilos.

5.3. Resultados algoritmo concurrente por procesos

En esta sección se presentan los resultados obtenidos al ejecutar el algoritmo concurrente por procesos. Se realizaron pruebas utilizando diferentes tamaños de matrices y se midió el tiempo de ejecución para cada caso. Para este caso se aplicó la optimización por compilador usada también en el algoritmo secuencial, y se compararon los resultados con la versión sin optimización. A continuación, se muestran las tablas con los tiempos de ejecución y las gráficas correspondientes para analizar el rendimiento del algoritmo concurrente por procesos.

Procesos (fork) sin optimizar					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	156,000	1.375,527	11.178,613	109.797,210	1.013.929,815
2	155,131	1.375,719	11.179,959	109.817,020	1.014.551,176
3	155,475	1.375,596	11.207,229	109.858,152	1.014.289,934
4	154,892	1.377,276	11.202,971	109.860,838	1.014.627,613
5	155,996	1.376,703	11.199,416	109.840,789	1.014.479,146
6	154,840	1.372,566	11.141,053	109.755,692	1.013.934,964
7	153,723	1.372,336	11.188,040	109.808,912	1.012.823,057
8	153,982	1.375,187	11.165,674	109.762,283	1.012.780,727
9	154,943	1.372,860	11.231,768	109.754,716	1.015.600,567
10	154,237	1.372,483	11.201,192	109.730,494	1.013.644,586
Promedio	154,922	1.374,625	11.189,591	109.798,611	1.014.066,159
Desv. Est.	0,739	1,783	23,757	44,013	808,497
CV (%)	0,48	0,13	0,21	0,04	0,08

Figura 10: Tiempos de ejecución del algoritmo concurrente por procesos para diferentes tamaños de matrices.

Procesos (fork) con O3_full					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	33,839	354,454	2.883,813	29.722,976	395.594,665
2	33,963	354,565	2.845,304	29.656,377	389.767,303
3	33,742	354,141	2.844,864	29.600,246	387.207,331
4	33,984	354,099	2.841,819	29.914,070	388.915,012
5	33,791	354,114	2.844,704	29.644,970	389.624,829
6	34,191	354,250	2.838,142	29.594,008	385.969,145
7	33,896	353,910	2.839,103	29.548,142	392.882,188
8	34,035	354,470	2.839,972	29.509,764	396.536,102
9	33,996	354,348	2.839,421	29.603,075	388.528,606
10	34,029	353,600	2.858,064	29.510,150	383.824,298
Promedio	33,947	354,195	2.847,521	29.630,378	389.884,948
Desv. Est.	0,126	0,276	13,274	113,440	3.846,020
CV (%)	0,37	0,08	0,47	0,38	0,99

Figura 11: Tiempos de ejecución del algoritmo concurrente por procesos con optimizaciones.

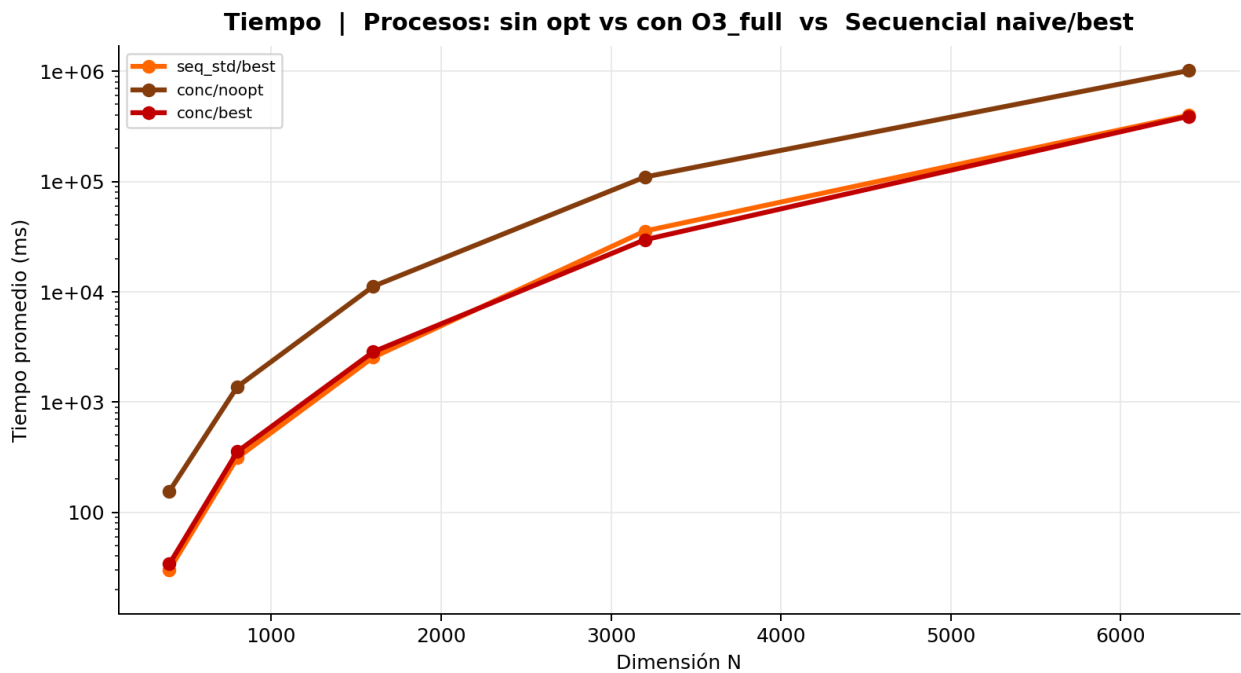


Figura 12: Gráfica de tiempos de ejecución del algoritmo concurrente por procesos.

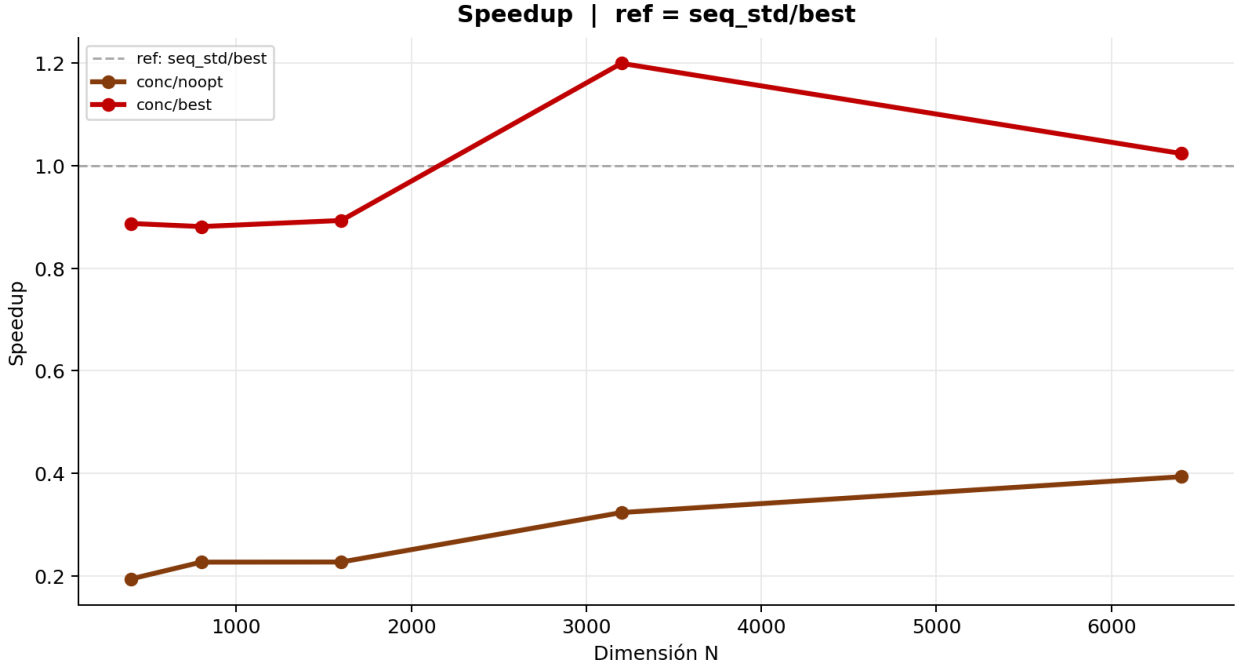


Figura 13: Gráfica de Speedup del algoritmo concurrente por procesos.

En este caso se puede observar que el algoritmo concurrente por procesos no muestra una mejora significativa en el tiempo de ejecución en comparación con el algoritmo secuencial, especialmente para tamaños de matrices pequeñas. Aún si observamos la el speedup del algoritmo concurrente por procesos, no se aprecia una mejora en el rendimiento, lo que sugiere que la sobrecarga de crear y gestionar procesos puede estar afectando negativamente el rendimiento del algoritmo. Sin embargo, es importante destacar que para tamaños de matrices más grandes, el algoritmo concurrente por procesos podría mostrar una mejora en el rendimiento, aunque esto no se refleja claramente en los resultados obtenidos en este caso.

5.3.1. Resultados del algoritmo secuencial optimizado por cache line

En esta sección se presentan los resultados obtenidos al ejecutar el algoritmo secuencial optimizado por cache line. Se realizaron pruebas utilizando diferentes tamaños de matrices y se midió el tiempo de ejecución para cada caso. Para este caso también se aplicó la optimización por compilador usada en el algoritmo secuencial, y se compararon los resultados con la versión sin optimización. A continuación, se muestran las tablas con los tiempos de ejecución y las gráficas correspondientes para analizar el rendimiento del algoritmo secuencial optimizado por cache line.

Cache line sin optimizar					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	149,266	1.164,373	9.324,405	74.720,618	598.025,731
2	145,776	1.164,345	9.338,473	74.749,358	599.668,009
3	146,841	1.166,793	9.330,197	74.686,730	599.701,999
4	148,514	1.168,687	9.332,290	74.683,145	599.904,723
5	145,732	1.166,323	9.327,693	74.707,249	599.659,017
6	146,931	1.165,958	9.328,194	74.756,151	599.633,795
7	146,686	1.167,340	9.332,661	74.740,884	599.626,095
8	146,543	1.166,599	9.336,166	74.683,460	599.310,805
9	146,725	1.168,452	9.340,401	74.683,827	597.652,492
10	147,345	1.169,849	9.334,826	74.695,346	599.768,751
Promedio	147,036	1.166,872	9.332,531	74.710,677	599.295,142
Desv. Est.	1,051	1,695	4,791	27,596	746,125
CV (%)	0,71	0,15	0,05	0,04	0,12

Figura 14: Tiempos de ejecución del algoritmo secuencial optimizado por cache line para diferentes tamaños de matrices.

Cache line con O3_full					
Rep	N = 400	N = 800	N = 1,600	N = 3,200	N = 6,400
1	17,570	140,398	1.125,407	9.301,438	90.183,945
2	17,653	140,422	1.125,699	9.299,227	90.387,316
3	17,782	140,583	1.127,719	9.313,929	90.319,322
4	17,730	140,328	1.125,894	9.303,453	90.395,212
5	17,683	140,551	1.125,675	9.299,517	90.442,760
6	17,563	140,374	1.124,918	9.296,268	90.425,732
7	17,546	152,669	1.125,383	9.296,248	90.335,517
8	17,836	140,455	1.124,912	9.286,314	90.256,993
9	17,661	140,462	1.124,498	9.287,268	90.301,539
10	17,646	140,289	1.124,897	9.288,838	90.260,445
Promedio	17,667	141,653	1.125,500	9.297,250	90.330,878
Desv. Est.	0,090	3,673	0,850	7,970	78,654
CV (%)	0,51	2,59	0,08	0,09	0,09

Figura 15: Tiempos de ejecución del algoritmo secuencial optimizado por cache line con optimizaciones.

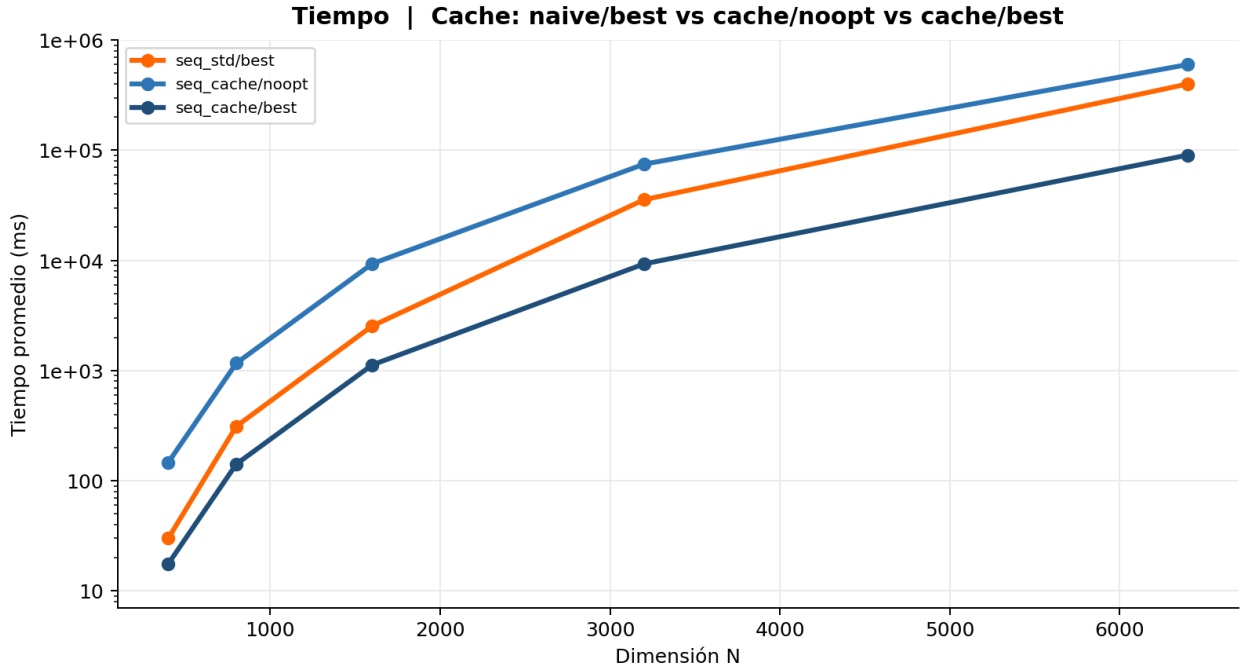


Figura 16: Gráfica de tiempos de ejecución del algoritmo secuencial optimizado por cache line.

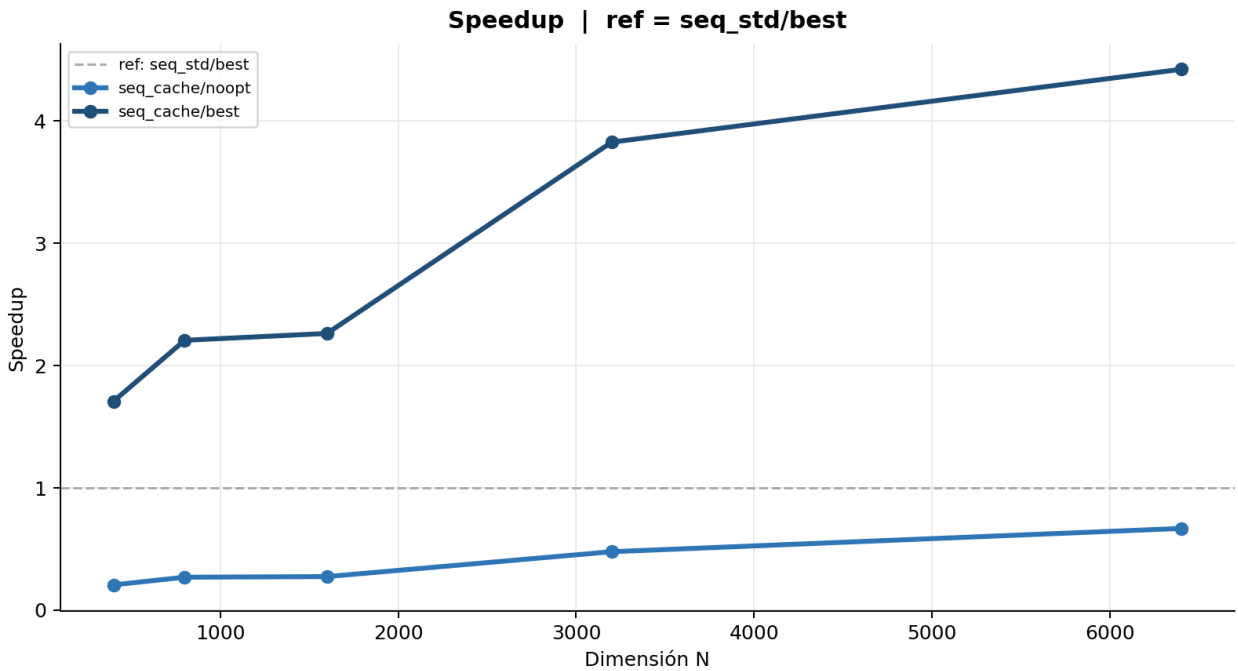


Figura 17: Gráfica de Speedup del algoritmo secuencial optimizado por cache line.

Para este caso se puede observar que el algoritmo secuencial optimizado por cache line no muestra una mejora significativa en el tiempo de ejecución en comparación con el algoritmo

secuencial con optimización por compilador. Sin embargo, el algoritmo optimizado por cache line con optimización por compilador muestra una mejora en el rendimiento, especialmente para tamaños de matrices más grandes, esto se puede observar en la gráfica de Speedup, donde se aprecia una mejora en el rendimiento del algoritmo optimizado por cache line con optimización por compilador en comparación con el algoritmo secuencial con optimización por compilador. Esto sugiere que la optimización por cache line puede ser efectiva para mejorar el rendimiento del algoritmo de multiplicación de matrices, especialmente para tamaños de matrices más grandes.

6. Comparaciones entre algoritmos

En esta sección se presentan las comparaciones entre los diferentes algoritmos implementados para la multiplicación de matrices: el algoritmo secuencial, el algoritmo concurrente con hilos, el algoritmo concurrente por procesos y el algoritmo secuencial optimizado por cache line. Se analizarán los tiempos de ejecución y el Speedup obtenido para cada algoritmo en función del tamaño de las matrices utilizadas en las pruebas. Es importante aclarar que en esta comparación solo se hace con los algoritmos que mejor resultado obtienen. A continuación, se muestran las tablas y gráficas correspondientes para comparar el rendimiento de cada algoritmo y discutir los resultados obtenidos.

5. Comparacion final						
Implementacion	N=400	N=800	N=1,600	N=3,200	N=6,400	Sp. prom.
Secuencial naive con O3_full	30.1 ms (ref)	312.1 ms (ref)	2,543.1 ms (ref)	35,540.4 ms (ref)	399,073.0 ms (ref)	1.00x
Concurrencia 6 hilos con O3_full	5.4 ms 5.62x	52.8 ms 5.91x	429.8 ms 5.92x	6,036.0 ms 5.89x	82,004.9 ms 4.87x	5.64x
Cache line con O3_full	17.7 ms 1.70x	141.7 ms 2.20x	1,125.5 ms 2.26x	9,297.2 ms 3.82x	90,330.9 ms 4.42x	2.88x
Procesos (fork) con O3_full	33.9 ms 0.89x	354.2 ms 0.88x	2,847.5 ms 0.89x	29,630.4 ms 1.20x	389,884.9 ms 1.02x	0.98x

Figura 18: Tabla de comparación de tiempos de ejecución entre los algoritmos secuencial, concurrente con hilos, concurrente por procesos y secuencial optimizado por cache line.

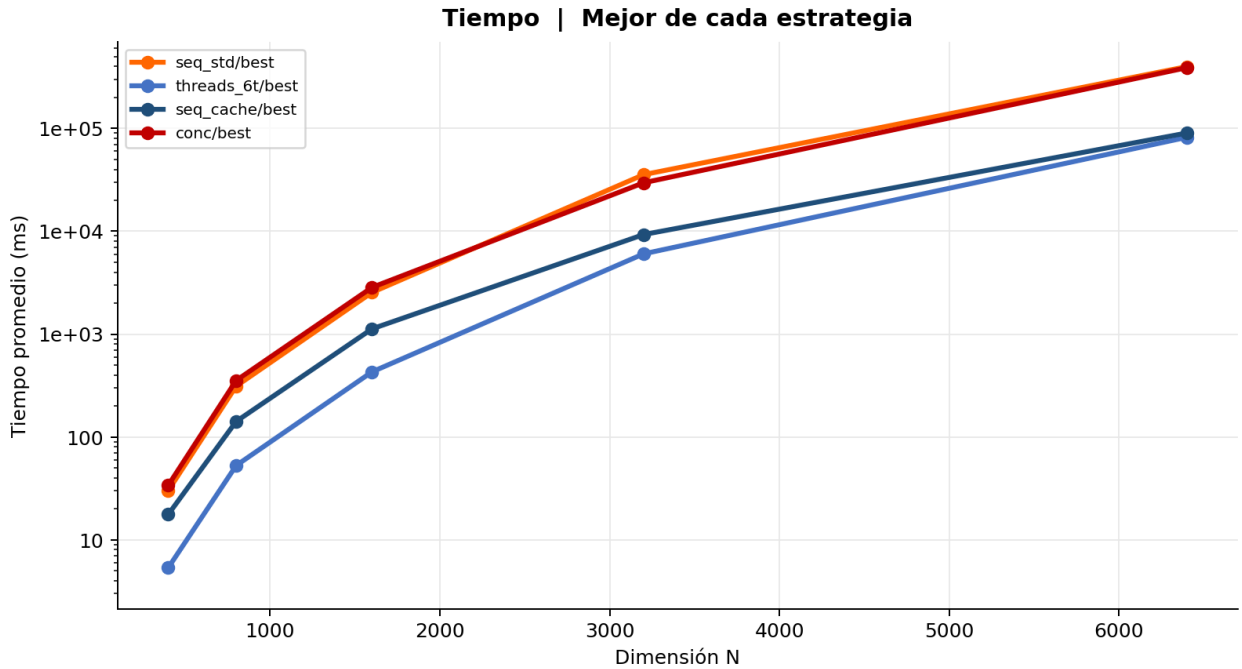


Figura 19: Gráfica de tiempos de ejecución comparativa entre los algoritmos secuencial, concurrente con hilos, concurrente por procesos y secuencial optimizado por cache line.

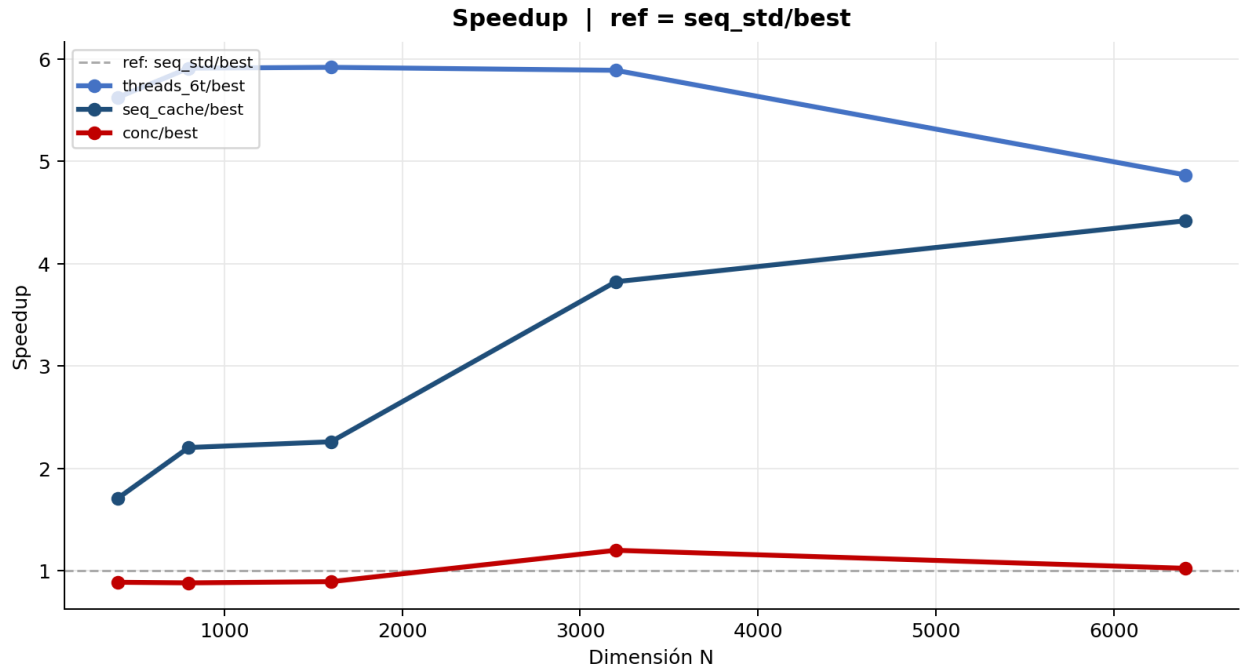


Figura 20: Gráfica de Speedup comparativa entre los algoritmos secuencial, concurrente con hilos, concurrente por procesos y secuencial optimizado por cache line.

Observando la tabla del speedup se puede apreciar que el algoritmo concurrente con 6 hilos

es el que mayor promedio de speedup tiene, seguido por el algoritmo secuencial optimizado por cache line. El algoritmo concurrente por procesos no muestra una mejora significativa en el rendimiento en comparación con el algoritmo secuencial, lo que sugiere que la sobrecarga de crear y gestionar procesos puede estar afectando negativamente el rendimiento del algoritmo. En general, se puede concluir que el algoritmo concurrente con hilos es el más eficiente para la multiplicación de matrices en este caso, aunque se aprecia una caída de rendimiento en las matrices más grandes, mientras que el algoritmo secuencial optimizado por cache line también muestra una mejora en el rendimiento.

7. Conclusiones

En este trabajo se implementaron y analizaron diferentes algoritmos para la multiplicación de matrices: el algoritmo secuencial, el algoritmo concurrente con hilos, el algoritmo concurrente por procesos y el algoritmo secuencial optimizado por cache line. Se realizaron pruebas utilizando diferentes tamaños de matrices y se midió el tiempo de ejecución para cada caso, aplicando optimizaciones por compilador y comparando los resultados obtenidos.

1. El algoritmo concurrente con hilos proporcionó el mejor rendimiento general, llegando a acelerar el proceso más de 5x respecto a la versión base. Se observó que el punto óptimo se alcanza utilizando 6 hilos, lo cual coincide exactamente con la cantidad de núcleos físicos del procesador utilizado. Incrementar la cantidad de hilos más allá de este punto genera rendimientos decrecientes debido a la sobrecarga de gestión y contención de recursos.
2. La aplicación de banderas agresivas del compilador redujo de manera drástica el tiempo de ejecución para todos los casos al permitir vectorización y desenrollado de bucles. Al sumarle a esto la reestructuración del algoritmo con optimización de cache line, se logra mitigar fuertemente la latencia de memoria de la CPU, demostrando ser fundamental para matrices de gran escala.
3. La paralelización basada en procesos resultó ser el método más ineficiente, rindiendo incluso por debajo de la implementación secuencial estándar optimizada por compilador. Esto evidencia que el gran costo general del sistema operativo al tener que crear, gestionar y aislar la memoria para procesos independientes anula cualquier ventaja del cálculo en paralelo en este contexto.
4. Solo el algoritmo concurrente con hilos muestra grandes ahorros de tiempo en matrices pequeñas dada la alta sobrecarga inicial. Los beneficios reales del speedup y del uso

adecuado de la memoria caché se revelan masivamente en tamaños de matriz superiores a 1600 y alcanzan su pico en las de 6400×6400 .

5. Realmente ninguna de las optimizaciones por sí sola es suficiente para lograr un rendimiento sobresaliente. El algoritmo secuencial optimizado por cache line sin optimización por compilador apenas mejora respecto a la versión base, y el algoritmo concurrente con hilos sin optimización por compilador tampoco alcanza su máximo potencial. Es la combinación de todas estas técnicas lo que permite alcanzar los mejores resultados. Si solo analizáramos los resultados de cada optimización por separado, podríamos concluir erróneamente que ninguna de ellas es efectiva, cuando el propio algoritmo con optimización por compilador ya muestra una mejora significativa respecto a las otras.
6. El nulo beneficio del Hyper-Threading para este contexto: Pese a que el procesador contaba con 12 hilos lógicos (SMT), el rendimiento máximo se estancó en 6 hilos. Esto demuestra que la multiplicación de matrices, al ser una operación puramente limitada por la capacidad de las Unidades Lógico Aritméticas (ALU) y no por paradas (stalls) en la CPU, no se beneficia de hilos lógicos compitiendo por los mismos recursos físicos dentro del mismo núcleo.

Referencias

- Allen, R., & Kennedy, K. (2001). *Optimizing Compilers for Modern Architectures: A Dependence-based Approach*. Morgan Kaufmann.
- Ben-Ari, M. (2006). *Principles of Concurrent and Distributed Programming* (2nd). Addison-Wesley.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd). The MIT Press.
- Dongarra, J. J., Du Croz, J., Duff, I. S., & Hammarling, S. (1990). A Set of Level 3 Basic Linear Algebra Subprograms. *ACM Transactions on Mathematical Software*, 16(1), 1-17. <https://doi.org/10.1145/77626.79170>
- Drepper, U. (2007). *What Every Programmer Should Know About Memory* (Technical Report). Red Hat, Inc. <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>
- Grama, A., Gupta, A., Karypis, G., & Kumar, V. (2003). *Introduction to Parallel Computing* (2nd). Addison-Wesley.
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532-533. <https://doi.org/10.1145/42411.42415>
- Hager, G., & Wellein, G. (2010). *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- Lam, M. S., Rothberg, E. E., & Wolf, M. E. (1991). The Cache Performance and Optimizations of Blocked Algorithms. *ACM SIGARCH Computer Architecture News*, 19(2), 63-74. <https://doi.org/10.1145/115953.115963>
- Patterson, D. A., & Hennessy, J. L. (2017). *Computer Architecture: A Quantitative Approach* (6th). Morgan Kaufmann.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th). Wiley.
- Strang, G. (2016). *Introduction to Linear Algebra* (5th). Wellesley-Cambridge Press.
- Yi, L., Li, C., & Guo, J. (2020). CPI for Runtime Performance Measurement: The Good, the Bad, and the Ugly. *IEEE International Symposium on Workload Characterization (IISWC 2020)*, 15-25. <https://doi.org/10.1109/IISWC50251.2020.00011>